
Project 13 — Text to Speech: Presentation Brief

Everything you need to present the DL project. Source repo: `~/dev/dl` (github.com/Dmitri1S9/DL). Slide deck + speaker notes live in `~/dev/dev_presentation/`. This document is the single source of truth behind those slides — read it once and you can answer anything in the Q&A.

1. The assignment (what was asked)

- **Course:** 192.151 *Introduction to Deep Learning*, 2026S, TU Wien.
- **Project 13: Text to Speech.** Build a system that maps **text** → **audio**, and **evaluate** it.
- **Pick one pipeline:**
 - **Option A** — FastSpeech2 (text → mel) + HiFi-GAN (mel → waveform). *Two stages.*
 - **Option B** — Tacotron2 (autoregressive text → mel) + neural vocoder. *Two stages.*
 - **Option C** — **VITS** (end-to-end text → waveform, latent-variable modeling). *One stage.*
- **Required deliverables (from the brief):**
 - Train the chosen model and generate audio on a **held-out test set**.
 - Report **training budget**: epochs/steps, batch size, **total GPU-hours**.
 - A small **qualitative gallery** (a few prompts) + discuss **failure modes** (mispronunciation, prosody, artifacts).
 - **Evaluation**: intelligibility proxy via off-the-shelf **ASR** → **WER/CER** (lower better); at least one spectral/prosody metric — **MCD** or F0 RMSE.

- (Optional) tokenizer study: character vs. phoneme tokens.
- Group of 3: **project report + group presentation + commented source code.**

We chose Option C (VITS).

2. The headline decisions (the story to tell)

1. **We pivoted A → C.** We began on the SpeechT5 + HiFi-GAN plan (Option A, two models: acoustic model + separate vocoder) and migrated to **VITS** (Option C, one model). Reasons: a single end-to-end model instead of two to train and keep in sync, and **noticeably better out-of-the-box audio**.
 2. **We fine-tune, not train from scratch.** We start from `kakao-enterprise/vits-ljs` — a strong LJSpeech VITS checkpoint — and adapt it to a target voice in a few GPU-hours. This is *real* training (we report epochs / batch size / GPU-hours) but realistic for a course compute budget.
 3. **We created our own fine-tuning dataset — a “B1 Battle Droid” voice.** This makes the before/after **audible**, not just a table of numbers (see §4).
 4. **We built the VITS training loop from scratch.** Hugging Face ships `VitsModel` as **inference-only** — there is no training forward pass, no MAS, no losses, no discriminator. We implemented all of it on top of the HF modules (text encoder, posterior encoder, flow, decoder). This is the core engineering contribution (see §6).
-

3. What VITS is (the model)

VITS = *Variational Inference with adversarial learning for end-to-end TTS* (Kim, Kong & Son, ICML 2021). It is a **single** network that goes straight from text to a raw waveform — no separate vocoder, no intermediate mel-spectrogram to predict and then re-synthesize.

It combines four ideas:

- **Conditional VAE (CVAE)** — a latent representation `z` of the speech.
- **Normalizing flow** — makes the prior over `z` more expressive.
- **Adversarial learning (GAN)** — HiFi-GAN-style discriminators push the decoder toward realistic audio.

- **Stochastic duration predictor** — models the one-to-many nature of speech (the same text can be spoken with different rhythms).

Components (and when each runs):

Component	Role	Train	Inference
Text encoder	text → prior distribution over z (mean, log-var)	✓	✓
Posterior encoder	linear spectrogram → posterior z	✓	—
Flow	maps posterior z [?] flow space z_p (invertible)	✓	✓
(Stochastic) duration predictor	how long each text token lasts	✓	✓
HiFi-GAN-style decoder	z → waveform	✓	✓
Discriminators (MPD + MSD)	real vs. generated audio (training signal only)	✓	—

Train vs. inference asymmetry: at training time the posterior encoder sees the *real* audio and gives a high-quality z ; the model learns to make the text-derived prior match it (KL term) and learns the text[?]audio alignment (MAS). At inference there is no audio, so we sample z from the prior, condition on predicted durations, and decode.

The alignment problem: text and audio frames are different lengths and we have no ground-truth alignment. VITS uses **Monotonic Alignment Search (MAS)** — a dynamic-programming search (Viterbi-like) for the highest-likelihood *monotonic* map from text tokens to audio frames. We implemented MAS from scratch (`alignment.py`).

4. The data story (the B1 Battle Droid voice)

- **Base dataset: LJSpeech** — single female English speaker, **13,100** clips (~24h). Standard single-speaker TTS benchmark. Last 500 clips are our held-out test set.
- **Fine-tuning target: a custom voice we built and published** — `Dm11tr13/ljspeech-b1` on the Hugging Face Hub (public, 13,100 examples, 22050 Hz).

- **Why a droid voice?** It makes the fine-tune’s effect **unmistakable to the ear** — you can *hear* the model move from the LJSpeech voice to the target voice. A subtle voice would make a demo ambiguous; “Star Wars B1 Battle Droid” is instantly recognizable.
- **How we built it (pipeline):**
 1. `data/bake_dataset.py` — synthesize all 13,100 LJSpeech transcripts with `vits-ljs` (clean speech, GPU, overnight, resumable).
 2. **B1 RVC voice conversion** (Docker, CUDA 12.8) — convert timbre to the droid voice with a *learned* B1 model (Homiebear/`B1BattleDroid`, `.pth` + FAISS `.index`; `index_rate 0.75`, `protect 0.33`, no pitch shift). RVC = Retrieval-based Voice Conversion — *changes timbre, not pronunciation*, so the transcripts stay valid.
 3. `data/prepare_b1_dataset.py` — resample 48 kHz → **22050 Hz** (to match VITS) and write `metadata.csv` (HF AudioFolder format).
 4. `data/push_b1_dataset.py` — push to the HF Hub.
- **Clever eval choice:** the *evaluation* test set stays on the **original LJSpeech audio** (same prompts), so before/after comparisons are apples-to-apples on identical text.

5. What we actually built (codebase tour)

```
src/
├─ core/          config · contracts · dto · logger          (shared infra)
├─ data/          download · prepare · prepare_training · bake_dataset
│                 prepare_b1_dataset · push_b1_dataset    (data + B1 pipeline)
├─ model/         synthesize.py                        (VITS inference + ckpt swap)
├─ vits_finetune/ config · model_config · dataset · collate · audio
│                 model · alignment · losses · discriminator
│                 checkpoint · train · synthesize        (★ the from-scratch trainer)
├─ evaluation/    metrics · evaluate                    (WER / CER / MCD)
├─ effects/       effects.py                            (fun voice FX — not in eval)
├─ tokenization/ tokenizer.py                          (char vs phoneme — stub)
├─ tests/         test_smoke.py                        (offline end-to-end wiring)
├─ test)
├─ scripts/       colab_setup.sh · sanity_check.py
├─ docs/          training-on-colab.md · Project13 brief PDF
├─ Makefile · requirements.txt (pinned) · pyproject.toml (ruff)
```

6. The core engineering: a from-scratch VITS trainer

Hugging Face’s `VitsModel` only does **inference**. To fine-tune it we re-implemented the entire training-mode forward pass and GAN machinery on top of its submodules. This is the part to dwell on in the talk.

6.1 `model.py` — `forward_train` (the training forward pass)

For each batch: 1. **Text encoder** → prior `mean`, `log-var`, hidden states. 2. **Posterior encoder** (real audio's linear spectrogram) → latent `z`. 3. **Flow** `z` → `z_p`. 4. **MAS alignment**: compute negative cross-entropy between `z_p` and the prior, run `maximum_path` (our MAS) → hard monotonic text→frame alignment `attn`. 5. **Duration loss** from the (stochastic) duration predictor against MAS durations. 6. **Random segment slice** of `z` (a fixed `segment_size` window) → **decoder** → predicted waveform; slice the matching real-waveform window as target. 7. Returns everything the loss needs: predicted/target waveform + mel, prior & posterior params, `z_p`, `z_mask`, duration loss.

(Segment slicing = the standard VITS/HiFi-GAN trick: the decoder is expensive, so train it on a short random crop, not the whole utterance.)

6.2 `alignment.py` — Monotonic Alignment Search (from scratch)

`neg_cross_entropy` builds the per-(token, frame) log-likelihood; `maximum_path` is a vectorized DP forward pass + traceback that finds the best monotonic alignment. `@torch.no_grad` — it's a search, not a learned module.

6.3 `losses.py` — five loss terms

- **Reconstruction** — L1 between predicted and target **mel** (weight **45**).
- **KL** — posterior || prior on the latent (weight **1**); the VAE term.
- **Duration** — from the stochastic duration predictor.
- **Adversarial (LSGAN)** — least-squares GAN loss for D and G.
- **Feature matching** — L1 between D's intermediate feature maps on real vs. fake (stabilizes GAN training).

6.4 `discriminator.py` — HiFi-GAN discriminators (from scratch)

- **MPD (Multi-Period Discriminator)** — reshapes 1-D audio into 2-D by periods `(2, 7, 13, 29, 37, 73, 97, 113, 137)`; each branch is a stack of 2-D convs. Catches **periodic structure** (pitch/harmonics).
- **MSD (Multi-Scale Discriminator)** — 1-D conv stacks at **3 scales** (audio progressively average-pooled). Catches structure at **different time scales**.
- Both return per-branch outputs **and feature maps** (for feature-matching loss).

6.5 `train.py` — the GAN training loop

- Two optimizers (**AdamW**), one for the generator (VITS), one for the discriminator.
- Per step: **(1) discriminator step** on real vs. detached-fake audio → `discriminator_loss`; **(2) generator step** → recon + KL + duration + adversarial + feature-matching.
- Periodic + end-of-epoch checkpoints (`epoch_N_G.pt` / `_D.pt`), resumable.
- Defaults: `batch_size=16` , `lr=2e-4` , `segment_size=16384` , 22050 Hz, 80 mels.

6.6 `synthesize.py` — inference + the before/after switch

Loads a checkpoint (or the pretrained base when omitted) and runs text → wav. The **only** difference between “before” and “after” audio is the `--checkpoint` flag → clean apples-to-apples comparison.

7. Evaluation methodology

- **WER / CER** (intelligibility): transcribe generated audio with **Whisper** (`base`) and compare to the reference text via `jiwer` . Lower = more intelligible.
- **MCD** (spectral fidelity): **Mel Cepstral Distortion** via `pymcd` (WORLD mel-cepstral + DTW alignment) between generated and real audio. ~5–8 dB is typical for decent TTS. Lower = closer to natural.
- **Before/after** is produced by the checkpoint swap (§6.6); the same `manifest ( {id, text, ref_audio} )` feeds both synthesis and scoring (contract).

Baseline measured (pretrained VITS, sanity sample):

model	WER	CER	MCD
pretrained <code>vits-ljs</code>	5.5%	1.6%	6.38 dB

(Fine-tuned numbers come from the full training run — see Status.)

8. Software engineering practices (worth a slide)

- **Contract-first architecture.** Pipeline stages talk through small contracts in `src/core` (a `{id, text, ref_audio}` manifest; an `EvalResult` DTO), not each other's internals. This let three people build in parallel and kept the model **swappable** (the A→C migration touched almost nothing downstream).
- **Reproducibility:** pinned exact dependency versions; `Makefile` one-liners (`make data/prepare/train/generate/eval/smoke`); fixed seed.
- **Offline mock pipeline** (`make all`) + **smoke test** (`test_smoke.py`) — prove the wiring end-to-end in seconds with no downloads/GPU.
- **Quality gates:** `ruff` lint + format (single quotes, line length 88); `loguru` logging (no bare `print`).
- **Colab runbook** (`docs/training-on-colab.md`) for GPU training, with checkpoint-to-Drive save (Colab VMs are ephemeral).

9. Status (be honest in the talk)

Component	State
Data download / prepare / training-prep	✅ done
B1 dataset pipeline → published <code>Dmitry13/ljspeech-b1</code>	✅ done
VITS inference + checkpoint swap	✅ done
VITS fine-tuning loop (<code>forward_train</code> , MAS, 5 losses, MPD+MSD discriminator, GAN loop)	✅ implemented
GPU fine-tuning run — 3 epochs on an NVIDIA L4 (Colab Pro), best @ epoch 2	✅ trained (checkpoint ~1 GB + reproducible notebook + local HTML demo)
Evaluation (WER/CER/MCD + before/after compare)	✅ done
Final fine-tuned WER/CER/MCD table	🟡 still to tabulate
char-vs-phoneme ablation	optional / stub (model itself already uses phonemes)

One-line honest summary: *the VITS fine-tuning machinery is built and it trained — 3 epochs on an L4, epoch 2 best, with a usable checkpoint. The discriminator was decisive (runs without it were unusable). What's left is the formal fine-tuned metrics table.*

Verified by synthesizing with the actual epoch-2 checkpoint (same line, base vs. fine-tuned): the fine-tune shifted spectral brightness **965** → **1364 Hz** (toward the RVC target ~1462) — so it *did* adapt toward the target voice —

but pitch stayed a human **~218 Hz with no ring-modulation**. **So it sounds like a brighter, thinner voice, not a hard “droid”** — because the RVC target itself is a converted human timbre, not the ring-mod robot effect (that lives in `effects.py`, not the training data). This is the honest answer to “it doesn’t sound like a droid”: the model adaptation worked; the *target* just isn’t a hard robot.

Note: `models/finetuned/training_log.json` is still a **placeholder mock** (references the old SpeechT5 plan) — don’t quote it as real results.

10. Failure modes to discuss (assignment requires this)

- **Mispronunciation** of rare words / names / numbers — grapheme→phoneme errors from espeak. (The model **already uses a phoneme tokenizer**, `vits-ljs`’s `VitsTokenizer` with `phonemize=True`; the repo’s `tokenization/` `char-vs-phoneme` module is a separate stubbed ablation, not the default.)
- **Prosody** — flat or odd rhythm; the stochastic duration predictor helps but isn’t perfect.
- **Artifacts** — buzzing/metallic GAN artifacts, especially early in training or on out-of-distribution text.
- **Fine-tuning risks** — too-high LR can wreck the pretrained checkpoint; GAN instability; over-fitting to the B1 timbre.

11. Team & contributions

Owner	Area	Files
Dima (Dmitrii Tsygankov)	model & fine-tuning	<code>src/model/</code> , <code>src/vits_finetune/</code>
Emir	infra, contracts, data pipeline	<code>src/core/</code> , <code>src/data/</code> , <code>Makefile</code>
Ilya	evaluation	<code>src/evaluation/</code>

32 commits, 30 May → 18 Jun 2026.

12. Likely discussion questions (and answers)

Q: Why VITS over FastSpeech2 / Tacotron2? One end-to-end model (no separate vocoder to train and keep in sync), better out-of-the-box quality, and latent-variable + adversarial modeling gives more natural prosody. Trade-off: more complex training (GAN + flow + MAS).

Q: Why fine-tune instead of training from scratch? From-scratch VITS needs days of multi-GPU training. Fine-tuning a strong LJSpeech checkpoint reaches a new target voice in a few GPU-hours — feasible on free Colab, still real training we can report.

Q: Why a synthetic (RVC) dataset and not real recordings? We needed a large *paired* (text, target-voice-audio) corpus. Recording 24h of a target voice is infeasible; RVC voice-converts the existing 13,100 LJSpeech clips into the target voice while preserving the transcripts — instant paired data.

Q: Doesn't training on VITS-then-RVC audio just teach the model its own artifacts? Partly a risk — it's a self-distillation-like setup. It's fine for demonstrating *voice adaptation* (the point here); for production you'd use real target-voice recordings.

Q: Why MCD and WER specifically? They're complementary: WER/CER measures *intelligibility* (can an ASR understand it?), MCD measures *spectral closeness* to the reference (does it sound like the target?). The brief asked for an ASR proxy + one spectral metric.

Q: What was the hardest part? Re-implementing VITS training on top of HF's inference-only `VitsModel`: MAS, the segment-slice decoder path, KL between posterior and aligned prior, and a stable GAN (MPD+MSD + feature matching).

Q: GPU-hours / budget? `batch_size=16`, `lr=2e-4`, `segment_size=16384`, 22050 Hz; default 2 epochs on Colab GPU (scale via `--num-epochs`). Exact GPU-hours come from the full run.

13. Demo assets (in repo)

- `audio/generated/test_000*.wav` — **pretrained-VITS** test-set outputs (real model).
- `droid_test.wav` — B1 droid voice sample.

- `audio/demo_1_normal.wav ... demo_5_*.wav` — **DSP voice effects** from `effects.py` (ring-mod robot filter + accent/emotion/stutter). **Post-processing, not model output** — keep this distinct from the RVC dataset voice in the talk.

Two “droid” paths in the repo — don’t conflate them. The *training dataset* (`Dm11tr13/ljspeech-b1`) is made by **RVC** (a learned voice). `effects.py` / `augmentation/magic.py` are an **earlier DSP-filter** approach + the demo clips, explicitly excluded from training/eval. Fine-tuning targets the RVC voice because a fixed DSP filter wouldn’t test whether the *model* learned anything.

14. Known code-level limitations (honest, for hard Q&A)

- **Mel length mismatch (~3 frames):** STFT uses `center=False`, so a 16384-sample decoder segment → 61 mel frames vs. the 64-frame target; `forward_train` takes `min(...)`, so it trains but truncates ~5% of the recon target. Fix: `center=True` or pad the segment.
- **Duration loss unweighted:** only mel (×45) and KL (×1) are configurable; duration / adversarial / feature-matching are added at an implicit weight of 1.
- **No fine-tuned metrics yet** — the GPU run is pending; `training_log.json` is a placeholder mock (still references the old SpeechT5 plan).
- **Tokenizer = phonemes** (espeak via `VitsTokenizer`, `phonemize=True`); the `tokenization/` char-vs-phoneme module is a stubbed ablation (identity functions).
- **Env grit:** Torch 2.11.0+cu128, Transformers 5.9.0 pinned; `espeak-ng` is a system dep; the RVC container needed CUDA/ `fairseq` / `omegaconf` pinning.
- Live: `make generate CKPT=... && make eval LABEL=...`, or `python -m vits_finetune.synthesize --checkpoint ... --text "Hello there."`